

Android Activity

1. *An Activity:* An Activity serves as the primary entry point for user interaction within an Android application. Each activity hosts a layout, which defines the User Interface (UI) elements that allow users to interact with the app.

Since every activity is associated with a layout, the Activity class acts as a bridge between the application's logic and its UI components. In other words, it provides the gateway through which users engage with the application, while also enabling developers to programmatically manage and respond to user interactions.

Just as programming languages like C, C++, Java, or Python have a specific entry point (main() method), an Android application has its entry point through the onCreate() method of the Activity class.

2. Key Characteristics of an Activity

- Represents one screen of the app.
- Acts as the entry point for user interaction.
- Each app must have at least one Activity (usually MainActivity).
- Can host and manage a layout (UI components).
- Activities communicate with each other using Intents.
- Controlled by the Activity Lifecycle (creation → running → paused → destroyed).

3. Types of Activities

- Main Activity – Entry point of the app.
- Secondary Activities – Other screens (e.g., profile, settings).
- Dialog Activity – Small pop-up like screen.
- Transparent Activity – Shows content with a transparent background (e.g., permission prompts).

4. *Android Activity Lifecycle*

An Activity Lifecycle defines the sequence of states that an Activity goes through, from the moment it is created until it is destroyed. These states are controlled by the Android operating system to ensure efficient memory management, smooth user experience, and proper resource handling.

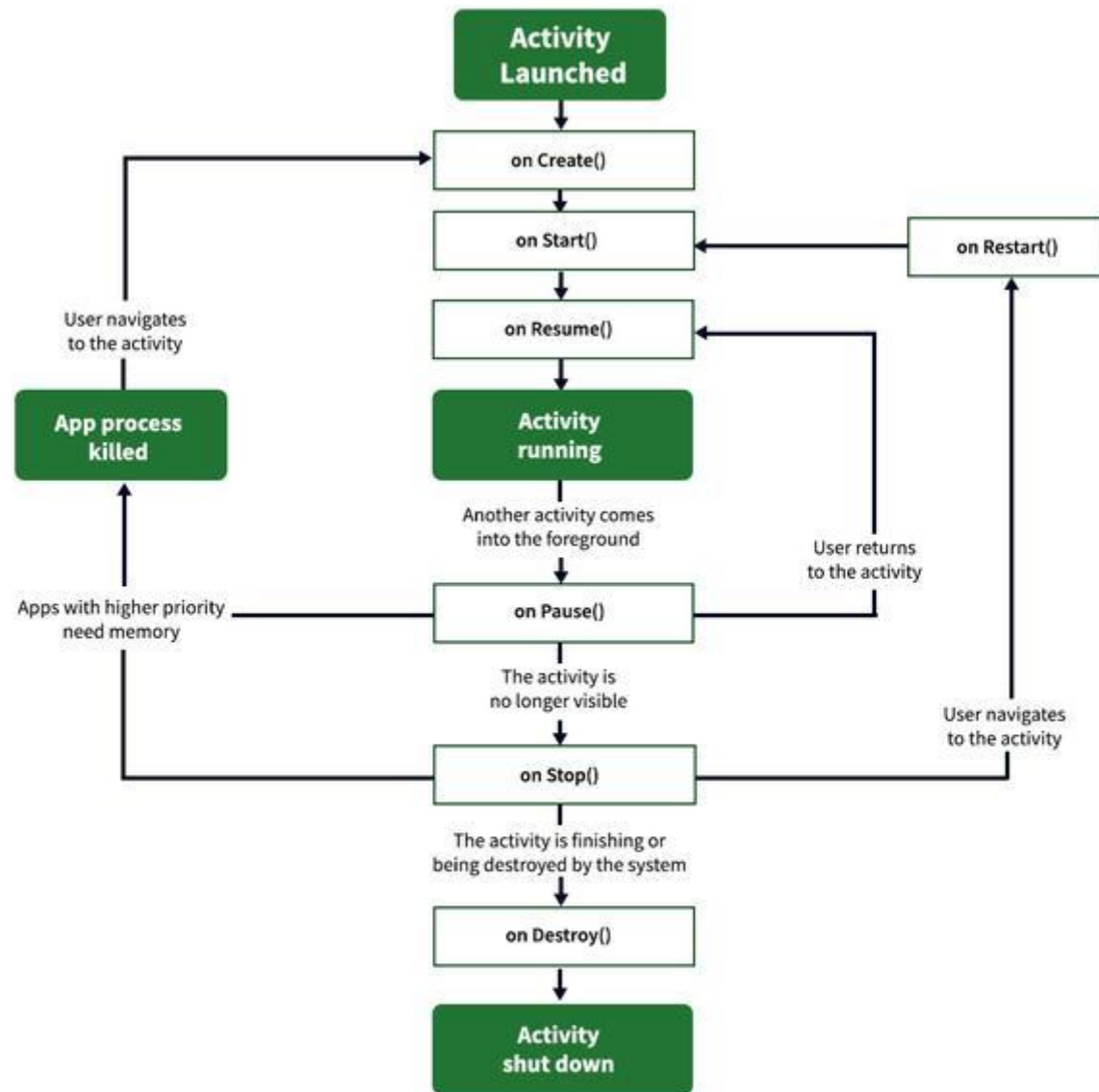
activities in Android do not run continuously, they are started, paused, stopped, and destroyed based on user actions, system requirements, or application logic.

States and Lifecycle: Each activity goes through various stages or a lifecycle and is managed by activity stacks. So when a new activity starts, the previous one always remains below it. There are four stages of an activity and 7 Important methods.

- a) Running: If an activity is in the foreground of the screen i.e at the top of the stack, then it is said to be active or running. This is usually the activity that the user is currently interacting with. With "running state" all other states are connected.
- b) Alive/ Pause: If an activity has lost focus and a non-full-sized or transparent activity has focused on top of your activity. In such a case either another activity has a higher position in multi-window mode or the activity itself is not focusable in the current window mode. Such activity is completely alive.
- c) Stopped or Hidden: If an activity is completely hidden by another activity, it is stopped or hidden. It still retains all the information, and as its window is hidden thus it will often be killed by the system when memory is needed elsewhere.
- d) Restart or Restore or Start: The system can destroy the activity from memory by either asking it to finish or simply killing its process. When it is displayed again to the user, it must be completely restarted and restored to its previous state.

5. *Methods in Android Activity Lifecycle*

The Activity Lifecycle in Android is controlled through callback methods provided by the Activity class. These methods allow developers to define how the application should behave during different states, such as creation, visibility, backgrounding, and destruction.



Activity Lifecycle in Android

1. *onCreate(Bundle savedInstanceState)*: onCreate method is called when the activity is first created. This is where all the static work is done like creating views, binding data to lists, etc. Equivalent to the main() method in other programming languages.
2. *onStart()*: onStart method is invoked when the activity is visible to the user. It is followed by onResume() if the activity is invoked from the background. It is also invoked after onCreate() when the activity is first started. The Activity is now visible to the user but not yet in the foreground. Suitable for lightweight tasks like refreshing UI elements. If interrupted, it may move to onStop().
3. onPause(): onPause method is invoked when an activity is going into the background but has not yet been killed. It is a counterpart to onResume(). When an activity is launched in front of another activity, this callback will be invoked on the top activity (currently on screen).
4. onResume(): onResume method is invoked when the activity starts interacting with the user. At this point, the activity is at the top of the activity stack, with a user interacting with it. Always followed by onPause() method when the activity goes into the background or is closed by the user. Called when the Activity comes to the foreground and becomes interactive. At this point, the user can fully interact with the UI.
5. onRestart() : OnRestart method is invoked after the activity has been stopped and prior to its starting stage and thus is always followed by onStart() when any activity is revived from background to on-screen.
6. onStop(): onStop method invoked when the activity is not visible to the user. It is followed by onRestart() when the activity is revoked from the background, followed by onDestroy() when the activity is closed or finished, and nothing when the activity remains on the background only. Note that this method may never be called, in low memory situations where the system does not have enough memory to keep the activity's process running after its onPause() method is called.
7. onDestroy(): The final call received before the activity is destroyed. This can happen either because the activity is finishing (when finish() is invoked) or because the system is temporarily destroying this instance of the activity to save space.

The Activity Lifecycle is like the journey of a human being:

- onCreate() → Birth
- onStart() → Childhood (visible but not yet active)
- onResume() → Adulthood (fully active)
- onPause() → Taking a break (Meditation)
- onStop() → Sleeping
- onRestart() → Waking up
- onDestroy() → Death (end of life cycle)

6. Common Interview/Exam Questions

- What is an Activity in Android?
- Explain the Activity Lifecycle with diagram.
- What is the difference between onCreate() and onStart()?
- Which method in the Activity Lifecycle is called first?
- Which method is similar to the main() method in Java or C++?
- In which method is setContentView() usually called?
- What is the purpose of the onCreate() method?